



Writeup — VaultDoorTraining (mot de passe en clair dans le code)

picoCTF · VaultDoorTraining · Reverse Engineering

Flag : `picoCTF{w4rm1ng_Up_w1tH_jAv4_0009yrGMeEp}`

🔗 Résumé de la faille

Le code source Java d'un programme de vérification de mot de passe est fourni. La fonction de vérification compare simplement l'entrée utilisateur à une chaîne **codée en dur** directement dans le code — le mot de passe est donc lisible à l'oeil nu, sans avoir besoin de "casser" quoi que ce soit.

Concept : le reverse engineering commence toujours par lire le code (quand il est disponible) avant de chercher des techniques plus complexes — et coder un secret en dur dans le code source est une faille de sécurité classique et réelle.

🔧 Méthodo

1. Récupérer et lire le fichier source

```
wget [URL]/VaultDoorTraining.java
cat VaultDoorTraining.java
```

Pas besoin de compiler ni d'exécuter le programme pour ce niveau "warmup" — la lecture directe du code source suffit.

2. Repérer la fonction de vérification

```
public boolean checkPassword(String password) {
    return password.equals("w4rm1ng_Up_w1tH_jAv4_0009yrGMeEp")
}
```

La comparaison se fait avec une chaîne **littéralement écrite dans le code** — c'est le mot de passe recherché, directement visible.

3. Comprendre le format attendu (bonus, pas nécessaire pour le flag)

```
String input = userInput.substring("picoCTF{".length(), userInput.length()-1);
```

Le programme retire lui-même les `picoCTF{` et `}` avant de comparer — donc si on voulait exécuter le programme, il faudrait taper le mot de passe encadré par ces accolades.

4. Le flag

Le mot de passe trouvé dans le code, remis dans le format `picoCTF{...}`, EST le flag :

```
picoCTF{w4rm1ng_Up_w1tH_jAv4_0009yrGMeEp}
```

💡 Le commentaire du challenge (la vraie leçon)

Le code contient ce commentaire, laissé volontairement par les créateurs du challenge :

"Is it safe to put the password in the source code? What if somebody stole our source code? Then they would know what our password is."

C'est LE point pédagogique de ce challenge d'introduction : coder un secret (mot de passe, clé API, clé de chiffrement...) **en dur dans le code source** est une mauvaise pratique de sécurité très répandue dans du vrai code de production. Quiconque a accès au code (dépôt Git, décompilation, fuite...) a accès au secret.

✅ Réflexes à retenir

- **Toujours lire le code source en premier** quand il est fourni, avant de sortir des outils de reverse plus lourds (Ghidra, gdb...). Beaucoup de "faux" challenges de reverse se résolvent à la simple lecture.
- Chercher les fonctions de type `checkPassword`, `verify`, `validate` ... et regarder à quoi elles comparent l'entrée.
- Un secret codé en dur dans le code est une vraie faille en production : les bonnes pratiques recommandent des variables d'environnement, un coffre-fort de secrets (vault), ou au minimum ne jamais commiter de secret dans un dépôt Git public.

🔒 Dans la vraie vie

Ce pattern (secret en dur dans le code) est une des causes les plus fréquentes de fuites de données réelles : des clés API AWS, des mots de passe de base de données, des tokens... retrouvés dans des dépôts GitHub publics ou dans des applications décompilées. Des outils comme **truffleHog** ou **gitleaks** scannent justement les dépôts à la recherche de secrets codés en dur oubliés dans l'historique Git.